

Universite Hassan II

Faculte des sciences et techniques de Mohammedia

Filiere ILISI

Meteo App

Application de Visualisation et de Previsions Meteorologiques

Rapport de projet

Module : Developpement Full Stack Moderne

Encadre par : Pr. Chakri Sanae

Realise par :

Bichri Yasser
Jounaid Ayoub

Annee universitaire :

2025 / 2026

Resume

Meteo App est une application Angular 21 complete qui illustre la mise en oeuvre des composants standalone, des Signals pour la gestion d'etat reactive, des formulaires reactifs, du routage parametre, du rendu SSR, et de l'integration d'API REST. L'application offre une experience utilisateur riche avec des visualisations SVG animees, une recherche de villes reactive, et une comparaison de donnees meteorologiques en temps reel.

Mots-cles : Angular 21, Signals, TypeScript, Tailwind CSS, SSR, Open-Meteo, SVG, Application meteo, Composants standalone.

Table des matières

Resume	1
1 Introduction	4
1.1 Presentation du projet	4
1.2 Technologies utilisees	4
1.3 Fonctionnalites de l'application	4
2 Architecture de l'application	5
2.1 Structure du projet	5
2.2 Routage	5
2.3 Composants et service partage	5
2.3.1 Diagramme d'architecture	6
3 Le service WeatherManager et la gestion d'etat	7
3.1 Presentation du service	7
3.2 Angular Signals : une gestion d'etat reactive	7
3.3 Diagramme de flux de donnees	7
3.4 Signal calcule (computed)	8
3.5 Gestion des appels API paralleles	8
4 Fonctionnalites detaillees	10
4.1 Recherche de villes avec autocompletion	10
4.1.1 Mecanisme	10
4.1.2 Le role du switchMap	10
4.1.3 Appel API	10
4.2 Previsions horaires (Hourly Forecast)	12
4.2.1 Recuperation des donnees	12
4.2.2 Affichage dans le template	12
4.2.3 Details d'implementation	12
4.3 Graphique des temperatures sur 5 jours	13
4.3.1 Principe general	13
4.3.2 Calcul des coordonnees	13
4.3.3 Structure du SVG	13
4.3.4 Animation du trace	14
4.4 Comparaison de deux villes	15
4.4.1 Architecture	15
4.4.2 Boutons de presets	15
4.4.3 Affichage des resultats	15
4.5 Qualite de l'air (AQI)	16
4.5.1 Donnees	16
4.5.2 Code couleur	16
4.5.3 Affichage	16
4.6 Favoris et villes recentes	17
4.6.1 Gestion des favoris	17
4.6.2 Villes recentes	17
4.7 Geolocalisation automatique	17

4.7.1	Mecanisme	17
4.8	Bascule d'unité de temperature	18
4.9	Mode sombre	18
4.10	Resume textuel (Weather Briefing)	19
5	Persistence des donnees	20
6	Rendu cote serveur (SSR)	21
7	Tests unitaires	22
8	Conclusion	23
8.1	Bilan technique	23
8.2	Defis techniques	23
8.3	Perspectives	23

Introduction

Presentation du projet

Meteo App est une application web de visualisation meteorologique developpee avec Angular 21 dans le cadre du module « Developpement Full Stack Moderne ». Elle permet a l'utilisateur de consulter les conditions meteorologiques actuelles, les previsions horaires sur 24 h, les tendances sur cinq jours avec un graphique dynamique, la qualite de l'air, et de comparer les donnees de deux villes cote-a-cote.

L'application exploite les API publiques d'Open-Meteo pour les donnees meteorologiques et de geocodage, ainsi que Nominatim (OpenStreetMap) pour le geocodage inverse. Le cœur du projet resider dans l'architecture Angular : composants standalone, gestion d'etat reactive avec les Signals, formulaires reactifs, routage parametre, et rendu cote serveur.

Technologies utilisees

- **Framework** : Angular 21 (standalone API, plus de NgModules)
- **Langage** : TypeScript 5.9
- **Gestion d'etat** : Angular Signals (`signal()`, `computed()`)
- **HTTP** : `HttpClient` d'Angular avec `RxJS`
- **Routage** : Router Angular (routes standalone)
- **Formulaires** : `ReactiveFormsModule` (`FormControl`)
- **Styles** : Tailwind CSS v4
- **SSR** : Angular SSR avec serveur Express
- **Tests** : Vitest + `HttpTestingController`
- **Icones** : Angular Material Icons
- **API externes** : Open-Meteo, Nominatim

Fonctionnalites de l'application

- Recherche de villes avec autocompletion et debounce
- Affichage des conditions meteorologiques actuelles (temperature, humidite, vent)
- Previsions horaires sur 24 h avec defilement horizontal
- Graphique SVG anime des temperatures sur 5 jours
- Indice de qualite de l'air (AQI) avec code couleur
- Comparaison de deux villes avec tableau et cartes jour par jour
- Gestion des favoris avec persistance locale
- Historique des dernieres recherches
- Geolocalisation automatique
- Bascule entre Celsius et Fahrenheit
- Mode sombre
- Resume textuel des conditions meteorologiques

Architecture de l'application

Structure du projet

L'application suit la structure standard d'un projet Angular avec des composants *standalone*. Tous les fichiers sources se trouvent dans le dossier `src/app/`.

Listing 1 – Arborescence du projet

```
src/
  app/
    app.ts                # Composant racine
    app.config.ts         # Configuration globale
    app.routes.ts         # Definition des routes
    app.routes.server.ts  # Modes de rendu SSR
    weather.ts            # Service metier et interfaces
    dashboard.ts          # Composant tableau de bord
    forecast.ts           # Composant previsions 5 jours
    compare.ts            # Composant comparaison
    *.spec.ts             # Tests unitaires
    styles.css            # Configuration Tailwind
    server.ts             # Serveur Express SSR
```

Routage

Les routes sont definies avec l'API standalone d'Angular :

```
1 export const routes: Routes = [
2   { path: '/', component: Dashboard },
3   { path: 'forecast/:lat/:lon/:name', component: Forecast },
4   { path: 'compare', component: Compare },
5   { path: '**', redirectTo: '' },
6 ];
```

Listing 2 – Definition des routes

Le routeur distingue trois vues principales :

- **Tableau de bord** (/) — page d'accueil avec la meteo actuelle, les previsions horaires et la qualite de l'air
- **Previsions detaillees** (/forecast/:lat/:lon/:name) — graphique SVG et cartes sur 5 jours, accessible depuis le bouton « Voir les previsions » du tableau de bord
- **Comparaison** (/compare) — comparaison de deux villes avec presets

La route `forecast` utilise `ActivatedRoute.paramMap` pour extraire les parametres dynamiques (latitude, longitude, nom). Elle est configuree en mode `RenderMode.Server` pour le SSR afin que les coordonnees dynamiques soient correctement rendues.

Composants et service partage

Tous les composants partagent un unique service `WeatherManager`, injecte via `providedIn: 'root'`, garantissant un etat coherent dans toute l'application.

Flux de données :

1. L'utilisateur interagit avec un composant (clic, saisie)
2. Le composant appelle une methode du WeatherManager
3. Le service effectue une requete HTTP vers une API externe
4. Les donnees recues mettent a jour un signal ()
5. Angular detecte le changement via sa detection de changements OnPush et re-rend la vue

Diagramme d'architecture

La figure 1 illustre l'architecture globale de l'application, montrant les trois couches principales : les composants Angular, le service central, et les API externes.

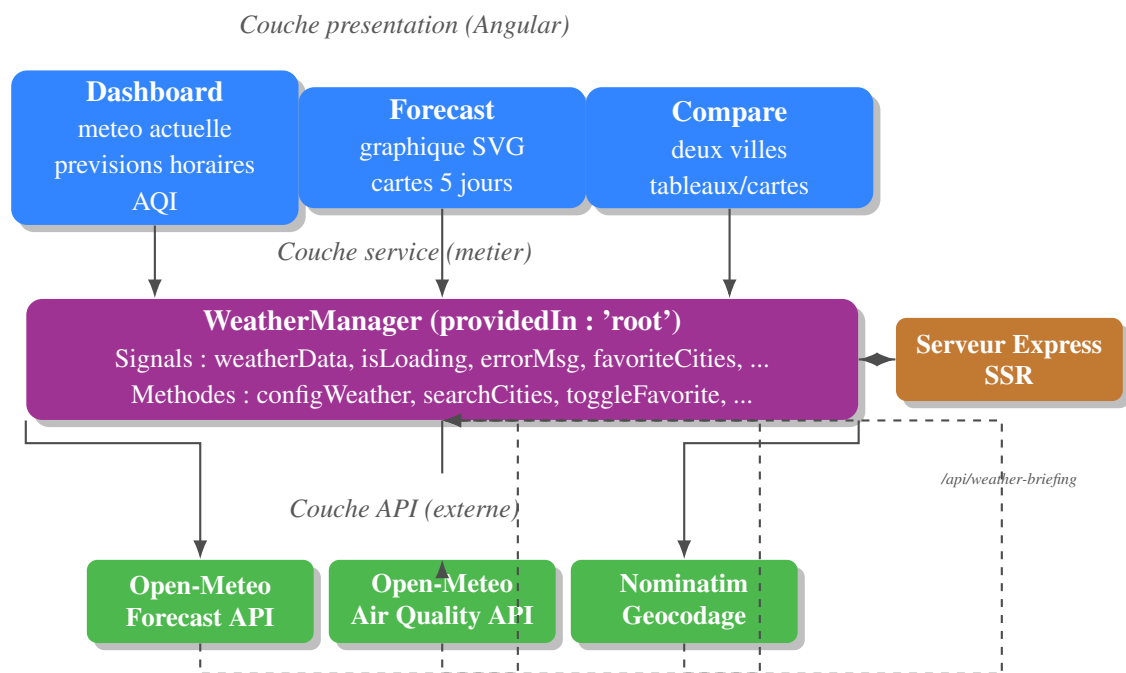


FIGURE 1 – Architecture de l'application Meteo App

Le service WeatherManager et la gestion d'état

Présentation du service

Le WeatherManager est le cœur métier de l'application. Declare avec `providedIn: 'root'`, il agit comme un singleton et centralise toute la logique : appels API, gestion des signaux, persistance `localStorage`, et transformations de données.

```

1 @Injectable({ providedIn: 'root' })
2 export class WeatherManager {
3   private http = inject(HttpClient);
4
5   // Signaux d'état
6   readonly temperatureUnit = signal<'C' | 'F'>('C');
7   readonly isDarkMode = signal<boolean>(false);
8   readonly currentCity = signal<City>(defaultCity);
9   readonly weatherData = signal<WeatherData | null>(null);
10  readonly isLoading = signal<boolean>(false);
11  readonly errorMsg = signal<string | null>(null);
12  readonly recentSearches = signal<City[]>([]);
13  readonly favoriteCities = signal<City[]>([]);
14  readonly compareCityA = signal<City | null>(null);
15  readonly compareCityB = signal<City | null>(null);
16  readonly compareDataA = signal<WeatherData | null>(null);
17  readonly compareDataB = signal<WeatherData | null>(null);
18  readonly isComparingLoading = signal<boolean>(false);
19  readonly aiSummary = signal<string | null>(null);
20  readonly isAiLoading = signal<boolean>(false);
21 }

```

Listing 3 – Declaration et signaux du service

Angular Signals : une gestion d'état reactive

L'application utilise exclusivement les Signals pour la gestion d'état. Ce choix permet :

- Une **reactivité fine** : Angular détecte précisément quel signal a changé et ne re-vérifie que le fragment du template concerne
- Une **gestion simplifiée** : pas de `BehaviorSubject`, `async pipe` ou `Subscription` à gérer pour l'état local
- Une **performance accrue** grâce au nouveau système de détection de changements
- **Aucune désinscription manuelle** : contrairement aux abonnements RxJS, les signaux ne nécessitent pas de `ngOnDestroy`

Diagramme de flux de données

La figure 2 illustre le parcours complet d'une interaction utilisateur, depuis la saisie dans le champ de recherche jusqu'à la mise à jour de l'interface.

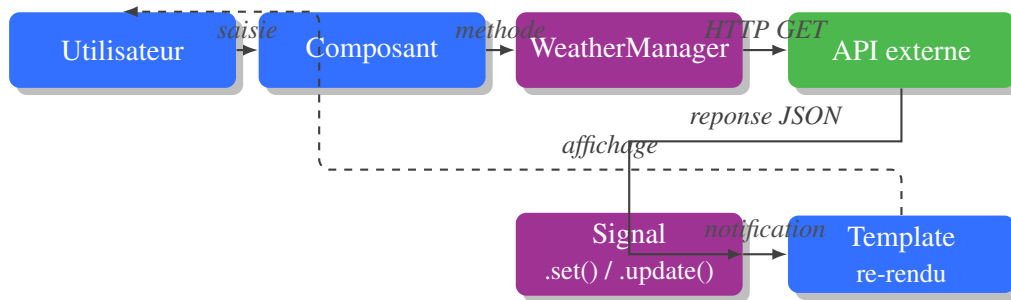


FIGURE 2 – Flux de données lors d'une interaction utilisateur

Signal calcule (computed)

Le signal derive `chartData` se recalcule automatiquement dès que `weatherData` change :

```

1 readonly chartData = computed(() => {
2   const data = this.weatherData();
3   if (!data) return null;
4   return this.computeChartCoordinates(data.forecast);
5 });

```

Listing 4 – Signal computed pour les coordonnées du graphique

Ce mecanisme garantit que les coordonnées SVG du graphique sont toujours synchronisees avec les données sans intervention manuelle.

Gestion des appels API paralleles

La methode `configWeather()` illustre le chargement parallèle de deux API via `forkJoin` :

```

1 configWeather(city: City): Observable<WeatherData> {
2   const params = `latitude=${city.latitude}
3     &longitude=${city.longitude}`;
4
5   const weather$ = this.http.get<OpenMeteoResponse>(
6     `https://api.open-meteo.com/v1/forecast?${params}
7     &current=temperature_2m,relative_humidity_2m,...`
8   );
9
10  const aqi$ = this.http.get<AqiApiResponse>(
11    `https://air-quality-api.open-meteo.com/v1/air-quality?${params}
12    &current=european_aqi,us_aqi,pm2_5,pm10,ozone`
13  ).pipe(catchError(() => of(null)));
14
15  return forkJoin({ weather: weather$, aqi: aqi$ }).pipe(
16    map(({ weather, aqi }) => this.mapToWeatherData(weather, aqi,
17      city))
18  );

```

Listing 5 – Chargement parallele des API meteo et qualite de l'air

Le `catchError(() => of(null))` sur l'API AQI est un choix delibere : si la qualite de l'air est indisponible, l'application continue de fonctionner et affiche simplement les donnees meteorologiques.

Fonctionnalites detaillees

Recherche de villes avec autocompletion

Mecanisme

La recherche de villes est implementee dans le composant Dashboard via un `FormControl` reactif. Le mecanisme repose sur une combinaison d'operateurs RxJS :

```
1 ngOnInit() {
2   this.sub = this.searchCtrl.valueChanges.pipe(
3     debounceTime(350),           // Attend 350ms apres la derniere
      frappe
4     distinctUntilChanged(),      // Ignore les valeurs identiques
5     switchMap(value => {
6       if (!value || value.trim().length < 2) {
7         return of([]);          // Pas de requete avant 2
          caracteres
8       }
9       this.isSuggesting.set(true);
10      return this.manager.searchCities(value);
11    })
12  ).subscribe(cities => {
13    this.suggestions.set(cities);
14    this.isSuggesting.set(false);
15  });
16 }
```

Listing 6 – Autocompletion avec debounce et switchMap

Le role du switchMap

L'operateur `switchMap` est essentiel : si l'utilisateur continue de taper avant que la requete ne revienne, il annule automatiquement la requete precedente et en emet une nouvelle. Cela evite :

- Les appels API superflus
- Les *race conditions* (affichage de suggestions pour une ancienne requete)
- La congestion reseau

Appel API

La methode `searchCities()` interroge l'API de geocodage d'Open-Meteo :

```
1 searchCities(query: string): Observable<City[]> {
2   return this.http.get<SearchResultItem>(
3     `https://geocoding-api.open-meteo.com/v1/search
4     ?name=${query}&count=10&language=en&format=json`
5   ).pipe(
6     map(response => response.results?.map(item => ({
7       id: item.id,
8       name: item.name,
9       latitude: item.latitude,
```

```
10     longitude: item.longitude,
11     country: item.country ?? '',
12     country_code: item.country_code,
13     admin1: item.admin1,
14   ))) ?? [])
15   );
16 }
```

Listing 7 – Requete de geocodage

Les resultats sont affiches dans un panneau deroulant positionne en absolu sous le champ de saisie. Chaque suggestion est cliquable et declenche `selectCity()` :

```
1 selectSuggestedCity(city: City) {
2   this.manager.selectCity(city);
3   this.suggestions.set([]);    // Vide les suggestions
4   this.searchCtrl.setValue('', { emitEvent: false });
5 }
```

Listing 8 – Selection d'une ville suggeree

Ce meme mecanisme est duplique dans le composant `Compare` pour les deux champs de recherche (Ville A et Ville B) de maniere independante.

Previsions horaires (Hourly Forecast)

Recuperation des donnees

Les previsions horaires sont incluses dans la requete principale a l'API Open-Meteo Forecast. Le parametre `hourly` demande les donnees heure par heure :

```
1 &hourly=temperature_2m,
2     precipitation_probability,
3     weather_code,
4     wind_speed_10m
```

Listing 9 – Parametres de la requete horaire

L'API retourne 24 valeurs (une par heure) pour chaque champ, stockees dans `data.hourly` sous forme de tableau `HourlyForecast[]`.

Affichage dans le template

Le template utilise une boucle `@for` avec defilement horizontal :

```
1 @if (data.hourly.length > 0) {
2   <div class="p-6 md:p-8 bg-white border-b border-slate-100">
3     <h3>Hourly Forecast</h3>
4     <div class="flex gap-3 overflow-x-auto pb-2 scrollbar-thin">
5       @for (hour of data.hourly; track hour.time) {
6         @let hCond = getCond(hour.weatherCode);
7         <div class="flex flex-col items-center gap-1.5
8             min-w-[64px] p-3 bg-slate-50 rounded-xl
9             border border-slate-100 shrink-0">
10          <span>{{ manager.formatHour(hour.time) }}</span>
11          <span class="material-icons {{ hCond.textColor }}">
12            {{ hCond.icon }}
13          </span>
14          <span>{{ formatTemp(hour.temperature, ...) }}</span>
15          @if (hour.precipitationProbability > 0) {
16            <span>{{ hour.precipitationProbability }}%</span>
17          }
18        </div>
19      }
20    </div>
21  </div>
22 }
```

Listing 10 – Template du bloc des previsions horaires

Détails d'implémentation

Plusieurs aspects techniques sont a noter :

- **Defilement horizontal** : la classe `overflow-x-auto` permet le scroll, et `scrollbar-thin` est un style personnalise defini dans `styles.css`
- **shrink-0** : chaque carte horaire conserve sa largeur minimale (`min-w-[64px]`) sans etre compressee

- **Icones dynamiques** : la fonction `getCond()` traduit le code meteorologique WMO en icone Material et couleur via l'objet `WeatherCondition`
- **Probabilite de precipitation** : affichee uniquement si superieure a 0%, pour ne pas surcharger l'interface

Graphique des temperatures sur 5 jours

Principe general

Le composant `Forecast` genere un graphique SVG inline montrant l'evolution des temperatures maximales (rouge) et minimales (bleu) sur 5 jours. Ce graphique est entierement calcule et dessine en TypeScript, sans bibliotheque de charting externe.

Calcul des coordonnees

La methode `computeChartCoordinates()` transforme les donnees numeriques en coordonnees pixels :

```

1 private computeChartCoordinates(list: DailyForecast[]) {
2   const width = 600, height = 240;
3   const paddingX = 50, paddingY = 40;
4   const count = list.length;
5   const xSpace = (width - paddingX * 2) / (count - 1);
6
7   const globalMax = Math.max(...list.map(d => d.tempMax));
8   const globalMin = Math.min(...list.map(d => d.tempMin));
9   let span = globalMax - globalMin;
10  if (span === 0) span = 1;
11
12  const pointsMax = list.map((d, i) => {
13    const x = paddingX + i * xSpace;
14    const y = height - paddingY -
15      ((d.tempMax - globalMin) / span) * (height - paddingY * 2);
16    return { x, y, val: d.tempMax, label: dayNames[...] };
17  });
18
19  const lineMax = pointsMax.reduce(
20    (p, pt, i) => i === 0
21      ? `M ${pt.x} ${pt.y}`
22      : `${p} L ${pt.x} ${pt.y}`, ''
23  );
24
25  return { pointsMax, pointsMin, lineMax, lineMin,
26    pathMaxArea, pathMinArea };
27 }
```

Listing 11 – Calcul des coordonnees SVG

Structure du SVG

Le SVG est construit avec les elements suivants :

- **Grille** : lignes horizontales en pointilles (`stroke-dasharray`) pour les repères de température
- **Lignes de température** : chemins `<path>` rouges (max) et bleus (min) avec une épaisseur de 3px
- **Aires remplies** : zones sous les courbes avec un dégradé (`<linearGradient>`) rose et bleu
- **Points de données** : cercles `<circle>` blancs avec bordure colorée
- **Étiquettes** : textes `<text>` pour les valeurs et les noms de jours

Animation du trace

L'animation utilise la technique du `stroke-dasharray` / `stroke-dashoffset` :

```
1 animateChart() {  
2   const svg = this.chartData();  
3   if (!svg) return;  
4   const length = Math.max(svg.lineMax.length * 3, 2000);  
5   this.chartDasharray = String(length);  
6   this.chartDashoffset = String(length);  
7  
8   requestAnimationFrame(() => {  
9     this.chartDashoffset = '0'; // Declenche l'animation  
10  });  
11 }
```

Listing 12 – Animation SVG avec `stroke-dasharray`

Le principe est le suivant :

1. Le chemin est initialement masqué avec `stroke-dashoffset` égal à la longueur totale
2. `requestAnimationFrame` modifie la valeur à 0
3. Le navigateur interpole et le chemin « se dessine » progressivement
4. Une fois terminée, les cercles des points apparaissent avec une animation d'échelle

Comparaison de deux villes

Architecture

Le composant `Compare` gere deux formulaires de recherche independants (Ville A et Ville B) et un ensemble de boutons de presets. Lorsque les deux villes sont selectionnees, les donnees sont chargees en parallele via `forkJoin` :

```
1 fetchComparisonDetails() {  
2   this.isComparingLoading.set(true);  
3  
4   forkJoin({  
5     dataA: this.configWeather(this.compareCityA()),  
6     dataB: this.configWeather(this.compareCityB()),  
7   }).subscribe({  
8     next: ({ dataA, dataB }) => {  
9       this.compareDataA.set(dataA);  
10      this.compareDataB.set(dataB);  
11      this.isComparingLoading.set(false);  
12    },  
13    error: () => this.isComparingLoading.set(false),  
14  });  
15 }
```

Listing 13 – Chargement parallele pour la comparaison

Boutons de presets

Des boutons de presets permettent de charger rapidement des paires de villes celebrees :

```
1 applyPreset('London', 51.5085, -0.1257, 'United Kingdom',  
2            'New York', 40.7143, -74.006, 'United States')
```

Listing 14 – Preset de comparaison

Chaque preset definit les deux villes avec leurs coordonnees exactes et declenche `fetchComparisonDetails`

Affichage des resultats

Les donnees sont presentees sous deux formes :

1. **Bandeau d'intelligence comparative** : affiche la difference de temperature entre les deux villes avec un message dynamique (« plus chaud », « plus froid », « identique »)
2. **Tableau de comparaison** : un tableau HTML semantique (`<table>`) avec les colonnes « Element atmospherique », « Ville A », « Ville B » et les lignes : Temperature, Ressenti, Condition, Humidite, Vent
3. **Cartes journalieres** : pour chaque jour, les temperatures max/min des deux villes sont affichees cote-a-cote avec un indicateur de difference

Qualite de l'air (AQI)

Donnees

L'indice de qualite de l'air est recuperee via l'API Open-Meteo Air Quality. Cinq indicateurs sont affiches :

```
1 &current=european_aqi,us_aqi,pm2_5,pm10,ozone
```

Listing 15 – Parametres AQI

Code couleur

La fonction `getAQIInfo()` attribue une couleur et un libelle a chaque valeur :

```
1 export function getAQIInfo(aqi: number | null) {
2   if (aqi === null) return { label: 'N/A', color: 'text-slate-400' };
3   if (aqi <= 20) return { label: 'Good', color: 'text-green
4     -500' };
5   if (aqi <= 40) return { label: 'Fair', color: 'text-lime-500
6     ' };
7   if (aqi <= 60) return { label: 'Moderate', color: 'text-yellow
8     -500' };
9   if (aqi <= 80) return { label: 'Poor', color: 'text-orange
10    -500' };
11   return { label: 'Very Poor', color: 'text-red-500' };
12 }
```

Listing 16 – Fonction de code couleur AQI

Affichage

Les donnees sont presentees dans une grille de 5 colonnes avec des cartes individuelles :

```
1 <div class="grid grid-cols-2 md:grid-cols-5 gap-3">
2   @for (item of aqiItems; track item.label) {
3     <div class="bg-slate-50 rounded-xl p-3 border
4       border-slate-100 text-center">
5       <span class="text-[9px] text-slate-400 uppercase font-bold">
6         {{ item.name }}
7       </span>
8       <span class="text-lg font-bold {{ item.color }}">
9         {{ item.value }}
10      </span>
11      <span class="text-[9px] text-slate-500 font-medium">
12        {{ item.unit }}
13      </span>
14    </div>
15  }
16 </div>
```

Listing 17 – Template de la grille AQI

Favoris et villes recentes

Gestion des favoris

Les favoris sont stockés dans `localStorage` sous la cle `meteo_favorites`. La methode `toggleFavorite()` gere l'ajout et le retrait :

```

1 toggleFavorite(city: City): void {
2   const current = this.favoriteCities();
3   const exists = current.some(
4     f => f.id === city.id ||
5         (f.latitude === city.latitude &&
6          f.longitude === city.longitude)
7   );
8
9   this.favoriteCities.set(
10    exists
11    ? current.filter(f => f.id !== city.id)
12    : [...current, city]
13  );
14
15  localStorage.setItem('meteo_favorites',
16    JSON.stringify(this.favoriteCities()));
17 }

```

Listing 18 – Ajout/retrait d'un favori avec persistance

La deduplication par `id` ou par coordonnees permet d'eviter les doublons, meme si la meme ville est selectionnee via differents moyens (recherche, preset, geolocalisation).

Villes recentes

Les villes recentes sont stockees sous la cle `meteo_recent_searches` avec un maximum de 6 entrees. La methode `addToRecents()` :

1. Charge les recents depuis `localStorage`
2. Supprime tout doublon eventuel (par `id` ou coordonnees)
3. Ajoute la nouvelle ville en tete de liste
4. Tronque a 6 elements
5. Persiste et met a jour le signal

Geolocalisation automatique

Mecanisme

La fonctionnalite « Detecter » utilise l'API Geolocation du navigateur combinee a un geocodage inverse via Nominatim (OpenStreetMap) :

```

1 triggerGeolocation(): Observable<City> {
2   // 1. Obtenir la position via l'API navigateur
3   return new Observable<GeolocationPosition>(observer => {
4     navigator.geolocation.getCurrentPosition(
5       pos => { observer.next(pos); observer.complete(); },
6       err => observer.error(err),

```

```

7      { enableHighAccuracy: true, timeout: 8000 }
8    );
9  }).pipe(
10    // 2. Inverser les coordonnees en nom de ville
11    switchMap(pos =>
12      this.http.get(
13        `https://nominatim.openstreetmap.org/reverse
14          ?lat=${pos.coords.latitude}
15          &lon=${pos.coords.longitude}
16          &format=json`)
17    ),
18    // 3. Transformer la reponse en objet City
19    map((res: any) => ({
20      name: res.address?.city
21        ?? res.address?.town
22        ?? 'Current Location',
23      latitude: parseFloat(res.lat),
24      longitude: parseFloat(res.lon),
25      country: res.address?.country ?? '',
26    })))
27  );
28 }

```

Listing 19 – Chaine complete de geolocalisation

Bascule d'unité de temperature

Le signal `temperatureUnit` stocke l'unité courante ('C' ou 'F') et est persiste dans `localStorage`. La conversion est effectuee par une fonction pure :

```

1 export function convertTemp(celsius: number, unit: 'C' | 'F'): number
2   {
3     return unit === 'F'
4       ? Math.round((celsius * 9) / 5 + 32)
5       : Math.round(celsius);
6   }
7 export function formatTemp(celsius: number, unit: 'C' | 'F'): string
8   {
9     return `${convertTemp(celsius, unit)}°${unit}`;
10  }

```

Listing 20 – Conversion de temperature

Tous les appels dans les templates utilisent `formatTemp(value, manager.temperatureUnit())` garantissant une coherence dans toute l'application.

Mode sombre

Le mode sombre est implemente via une classe CSS `.dark` appliquee dynamiquement sur `<html>` :

```
1 toggleDarkMode(): void {  
2   this.isDarkMode.update(d => !d);  
3   localStorage.setItem('meteo_dark_mode', String(this.isDarkMode()));  
4   this.applyDarkMode();  
5 }  
6  
7 applyDarkMode(): void {  
8   document.documentElement.classList.toggle('dark', this.isDarkMode()  
9   );  
}
```

Listing 21 – Bascule du mode sombre

La configuration Tailwind declare une variante dark custom basee sur cette classe. Chaque composant adapte son apparence via les classes `dark:bg-slate-800`, `dark:text-white`, etc. La preference est persister dans `localStorage`.

Resume textuel (Weather Briefing)

Après chaque selection de ville, le service appelle une route `/api/weather-briefing` via le serveur Express SSR. Cette route genere un court resume des conditions actuelles :

```
1 server.get('/api/weather-briefing', (req, res) => {  
2   const { city, temp, condition } = req.query;  
3   res.json({  
4     summary: `Presently ${temp} deg C with  
5             ${condition} in ${city}.`  
6   });  
7 });
```

Listing 22 – Route du serveur pour le resume textuel

Cette approche permet de generer le texte cote serveur, ouvrant la possibilite d'utiliser des algorithmes de traitement de langage naturel sans exposer de logique complexe au client.

Persistence des donnees

L'application utilise le `localStorage` du navigateur pour maintenir les preferences et donnees utilisateur entre les sessions. Quatre cles sont persistees :

TABLE 1 – Cles localStorage

Cle	Valeur	Declencheur
meteo_temp_unit	'C' ou 'F'	Bascule d' unite
meteo_dark_mode	'true' ou 'false'	Bascule mode sombre
meteo_recent_searches	City[] JSON (max 6)	Nouvelle selection de ville
meteo_favorites	City[] JSON	Ajout/retrait favori

Tous les acces a `localStorage` sont proteges par `typeof localStorage !== 'undefined'` pour la compatibilite avec le rendu serveur.

Rendu cote serveur (SSR)

L'application utilise Angular SSR avec un serveur Express. Deux modes de rendu sont configurés :

```
1 export const serverRoutes: ServerRoute[] = [  
2   {  
3     path: 'forecast/:lat/:lon/:name',  
4     renderMode: RenderMode.Server,    // Dynamique  
5   },  
6   {  
7     path: '**',  
8     renderMode: RenderMode.Prerender, // Statique  
9   },  
10  ];
```

Listing 23 – Configuration des modes de rendu SSR

La route `forecast` utilise le rendu serveur dynamique car ses parametres (coordonnees de la ville) sont inconnus au moment du build. Les autres routes sont pre-generees a la compilation grace au `Prerender`.

Tests unitaires

Les tests sont écrits avec Vitest, un framework de test rapide et compatible avec le CLI Angular. Les requêtes HTTP sont simulées via `HttpTestingController`.

```
1 it('should search cities via geocoding API', () => {
2   const mockResponse = {
3     results: [{
4       id: 1, name: 'Paris', latitude: 48.85,
5       longitude: 2.34, country: 'France', country_code: 'FR'
6     }]
7   };
8
9   service.searchCities('Paris').subscribe(cities => {
10     expect(cities.length).toBe(1);
11     expect(cities[0].name).toBe('Paris');
12   });
13
14   const req = httpMock.expectOne(req =>
15     req.url.includes('geocoding-api.open-meteo.com')
16   );
17   expect(req.request.method).toBe('GET');
18   req.flush(mockResponse);
19 });
```

Listing 24 – Test de la recherche de villes

Les tests couvrent :

- La création des composants et du service
- La bascule d'unité de température
- Le mode sombre
- L'ajout et le retrait de favoris
- Le nettoyage des villes récentes
- Les appels API simulés
- Les fonctions pures (`convertTemp()`, `formatTemp()`, `getWeatherCondition()`)

Conclusion

Bilan technique

Ce projet a permis de mettre en oeuvre l'ensemble des concepts fondamentaux d'Angular 21 :

- **Architecture standalone** : composants, routes et configuration sans NgModules
- **Gestion d'état reactive** : `signal()`, `computed()`, `detection OnPush`
- **Formulaires reactifs** : `FormControl`, `debounce`, `switchMap`
- **HTTP et RxJS** : `HttpClient`, `forkJoin`, `catchError`
- **Routage** : routes paramétrées, `ActivatedRoute`, navigation
- **SSR** : rendu serveur avec Express, modes Server et Prerender
- **Animations SVG** : calcul de coordonnées, `stroke-dasharray`, `requestAnimationFrame`
- **Persistance** : `localStorage` avec protection SSR
- **Tests** : Vitest, `HttpTestingController`
- **Integration API** : trois API REST externes orchestrées

Defis techniques

Les principaux défis rencontrés incluent :

- La coordination des appels API parallèles avec `forkJoin`
- L'animation du graphique SVG sans bibliothèque externe
- La gestion des états de chargement, d'erreur et de données vides (skeleton, erreur, contenu)
- La synchronisation des états entre les composants via le service partage
- Le rendu SSR avec des paramètres de route dynamiques

Perspectives

Plusieurs améliorations pourraient être apportées :

- Graphiques supplémentaires (précipitations, vent, pression atmosphérique)
- Internationalisation (i18n) avec `@angular/localize`
- Progressive Web App (PWA) avec `@angular/pwa`
- Tests de bout en bout avec Cypress ou Playwright
- Optimisations avec `defer` et chargement paresseux (*lazy loading*)